```
Output:
Come at me bro!
```

`function_exists()` accepts a single string argument and tests global scope for a function with the same name, returning boolean `true` if found (`false` if it isn't). By testing if `collision()` is already declared, we can skip declaring it again and killing our program. If `collision()` wasn't declared, the declaration inside the `if` block would have executed, and we'd have had the second version of the function in global scope. This particular trick is effective for avoiding runtime errors and historically was something of a necessity in PHP, when libraries installed with PEAR would commonly stomp all over your code (or each other's).

Another method of avoiding name collisions (prior to PHP 5.3) was to simply prefix the name you wanted to use with some other word, usually separated by an underscore or similar character. Thus, `logMessage()` might become `MyProject_logMessage()`, which is fine as far as it goes but does create issues of maintenance and readability (perhaps you already had several hundred calls to the original `logMessage()` before `ShinyLogs` became part of the project). The next section will show a more elegant way of mitigating name collision issues in global scope by using namespaces.

If for some reason you want to see *all* declared functions, there is the built-in `get_defined_functions()`, which will return an array of every single function registered when it is called. This can be rather large (a typical PHP installation will have thousands). Of more use might be `get_extension_functions()`, which will return every function for a named PHP extension:

```
print_r(get_extension_funcs('SPL'));
```

```
Output:
Array
(
    [0] => class_implements
    [1] => class_parents
    [2] => class_uses
    ...
```

A list of loaded extensions can be obtained via `get_loaded_extension()`.